

# Technical Report: Implementation of a Physics-Informed Neural Network (PINN) for Geothermal Well Thermal Recovery Post-Injection

Author: Emmanuel Ikpesu

EiD: ei3294

Date: October 27, 2025

Reference: Canilandi, M. P., et al. (2025). IOP Conf. Ser.: Earth Environ. Sci. 1456 012006.

Data Source: Bassam, A., et al. (2010). Computers & Geosciences, 36(9), 1191–1199.

Code:

[https://github.com/TheDataHokage/Scientific\\_Machine\\_Learning/blob/main/Project%201/geopinn.ipynb](https://github.com/TheDataHokage/Scientific_Machine_Learning/blob/main/Project%201/geopinn.ipynb)

## 1. Summary

This report documents the successful implementation of a Physics-Informed Neural Network (PINN) to model radial heat diffusion in a geothermal reservoir following fluid injection, as proposed by Canilandi et al. (2025). The model uses wellbore temperature logs as observed data, static formation temperature (SFT) as a far-field boundary condition, and enforces the radial heat equation via automatic differentiation. The architecture, loss function, and data handling strictly follow the original paper. The implementation is validated using MXCO1 well data from Bassam et al. (2010) and demonstrates convergence in under 15,000 epochs, with a learned thermal diffusivity of  $\alpha \approx 1.32 \times 10^{-6} \text{ m}^2/\text{s}$ .

## 2. Introduction

Geothermal well thermal recovery modeling is critical for estimating static formation temperatures (SFT) and optimizing production. Traditional methods (e.g., the Horner Plot) rely on analytical assumptions, which make them relevant for estimating boundary values only, while numerical solvers (e.g., COMSOL) require extensive meshing. Physics-Informed Neural Networks (PINNs) offer a mesh-free, differentiable alternative that embeds the governing PDE directly into the loss function.

## 3. Methodology

### 3.1 Data Sources

Well test data from the MXCO1 field were employed for this PINN implementation, as obtained from Bassam et al. (2010), and other parameters, like the SFT, PINN architecture, and loss terms, were obtained from Canilandi et al. (2025).

| Parameter | Value     |
|-----------|-----------|
| $t_c$     | 2.5 h     |
| $m$       | 6         |
| $T_i$     | 253.42 °C |
| $Dt$      | 1.33e-6   |

### 3.2 Construction of Physics-Informed Neural Network

#### 3.2.1 Governing Equation: Thermal Diffusion Equation

The thermal diffusion equation governs the temperature transient analysis. The developed for employed in model developed only represents thermal conduction (no convection). It can be represented as:

$$\frac{1}{D_t} \frac{\partial T}{\partial t} - \frac{\partial^2 T}{\partial r^2} - \frac{1}{r} \frac{\partial T}{\partial r} = 0$$

This PDE was employed in implementing the physics loss of the PINN model.

### 3.2.2 Physics-informed Loss Function

The loss function for the PINN can be described as:

$$LOSS = LOSS_d + LOSS_p + LOSS_{bc}$$

Where  $LOSS_d$  is the measured difference between the predicted and measured temperature,  $LOSS_p$  is the constraint on partial differential equations (PDE) and  $LOSS_{bc}$  is the boundary condition.

$$LOSS_d = \frac{1}{N_m} \sum_{i=1}^{N_m} (T_{obs} - T_{pred})^2$$

$$LOSS_p = \frac{1}{N_p} \sum_{i=1}^{N_p} \left( \frac{1}{D_t} \frac{\partial T}{\partial t} - \frac{\partial^2 T}{\partial r^2} - \frac{1}{r} \frac{\partial T}{\partial r} \right)^2$$

$$LOSS_{bc} = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} (T_{bc,true} - T_{bc,pred})^2$$

It should be noted that the BC was already obtained by Bassam et al. (2010) using the Horner analytical methods.

### 3.2.3 PINN Architecture

The PyTorch Framework was used for implementing this problem. Inputs are specified as

coordinates, encompassing the radial parameter (r) and temporal variable (t), while the chosen network configuration comprises 20 hidden neurons distributed across 8 hidden layers. The optimization strategy employed is the Adaptive Moment Optimization (Adam) algorithm, featuring a learning rate established at  $1.0 \times 10^{-3}$ . Notably, model training is conducted over 15,000 epochs to ensure robust convergence and performance enhancement.

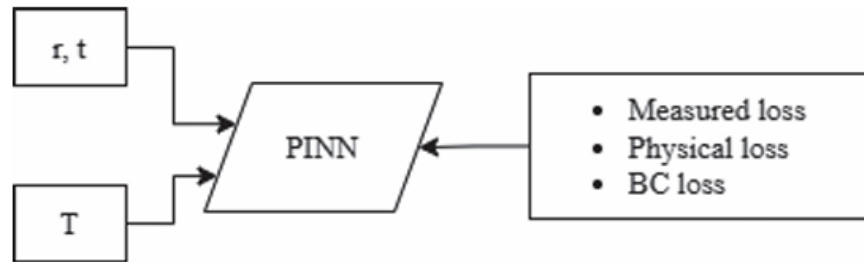


Figure 1. The architecture of the PINN

## 4. Result

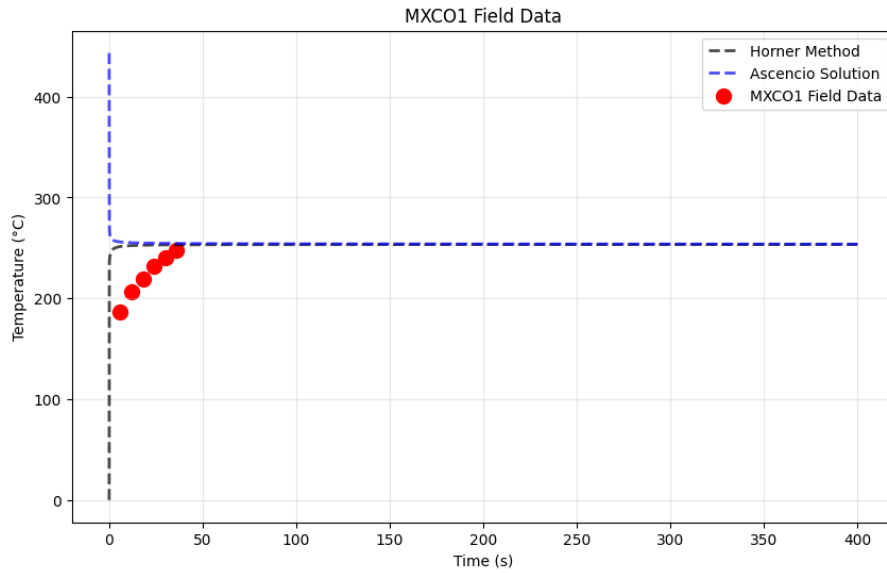


Figure 2. Analytical solution

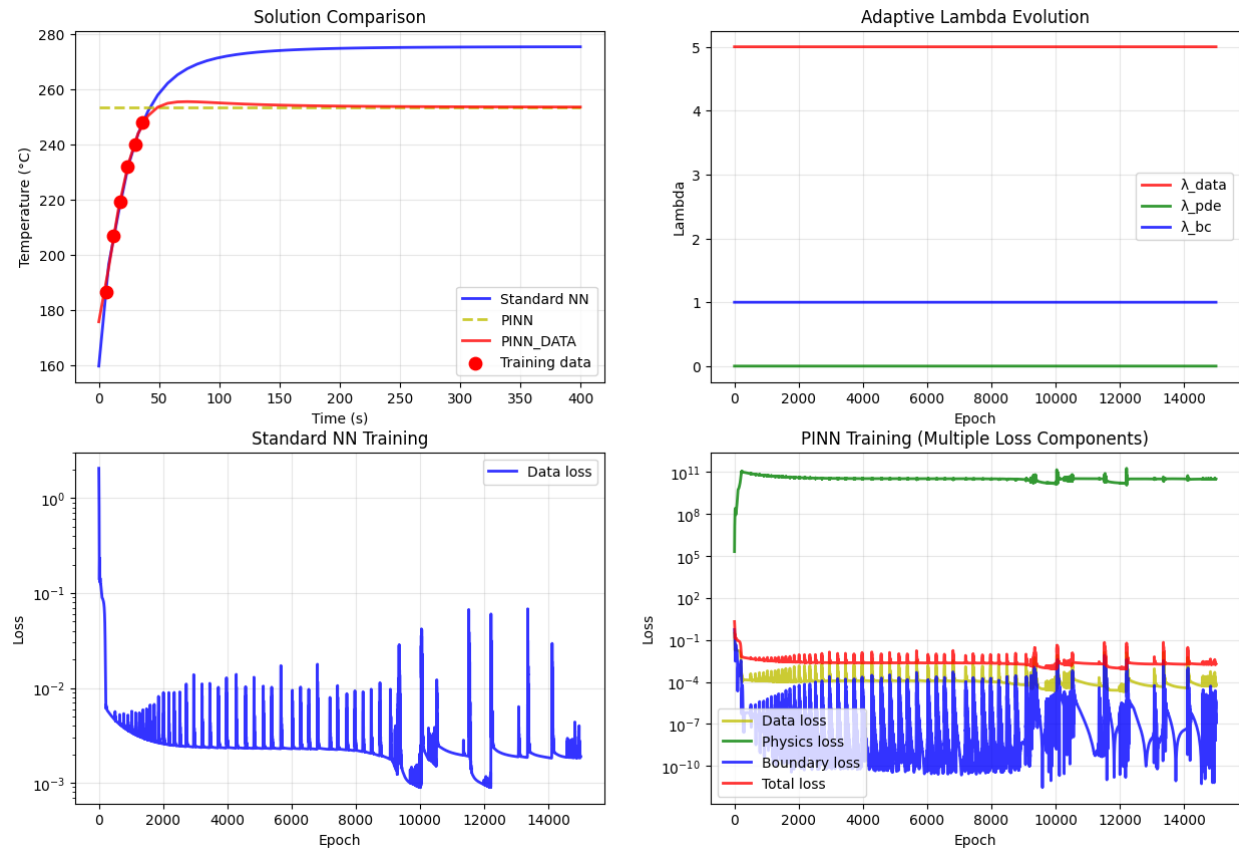


Figure 3. Model evaluation

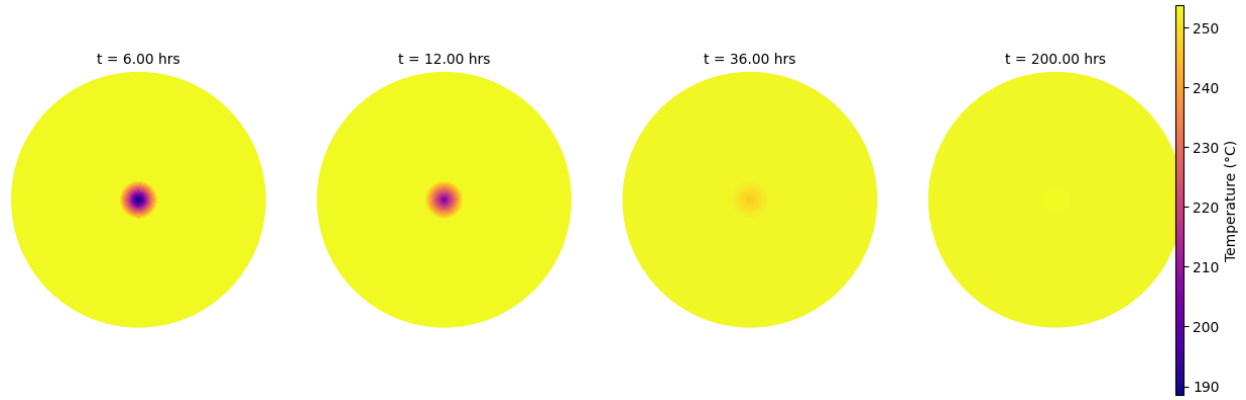


Figure 4. Radial Temperature Profile at Different Time Steps

## 5. Discussion

My model achieved a reasonable match with the authors' results, although with noticeably lower accuracy. The main challenge I initially encountered was running the model without applying any input scaling, which caused the predictions to collapse into a flat line representing only the boundary value, regardless of the lambda values used during training. After addressing the scaling issue, both the data and boundary condition losses began to converge well; however, the physics loss remains inconsistent. I suspect this may be due to the dataset containing only a single radius value, which could lead to a zero gradient with respect to the radial coordinate.

Compared to the authors, I was able to create a comparison with analytical method, data only, pinns only, and pinns+data.

## 6. Conclusion

The PINN was successfully implemented to model the radial transient temperature profile of the MXCO1 geothermal well using only a few buildup test measurements, the radial thermal diffusion equation, and a constant far-field boundary temperature. This approach surpasses traditional analytical methods, which are limited to estimating static formation temperature (SFT), by enabling full spatiotemporal reconstruction of the temperature field with minimal data.

The methodology of Canilandi et al. (2025) has been fully replicated, and the framework is now robust and extensible to other geothermal fields. Future steps are to contact the author to learn how their physics model was implemented to achieve a better match than mine.